The learning oracle

A self-contained statement of the framework, as agreed on 2026-08-22

2026-08-22

Contents

1 What this document is **1**

2 The problem the framework solves **2**

3 Notation **2**

4 Recipes and traces **3**

5 The learning oracle, passive form **3**

6 Views **4**

7 The learning oracle, interactive form **5**

8 Parties and the two kinds of game **7**

9 The adversary class **8**

10 Cost, and the ratio between parties **9**

11 The price of the search **9**

11.1 The price of aiming 9

11.2 The price of skipping is recipe-relative 10

11.3 An experiment: the measured exponent of the search 11

12 The corpus, mapped onto the framework **12**

13 The backdoor game, as far as it is fixed **13**

14 What is deferred, and what remains open about the oracle **15**

1 What this document is

This document states, from nothing, the formal framework that the survey's formalization section will rest on. It assumes the reader knows game-based cryptography and machine learning, and

assumes no prior exposure to the framework itself. Every object is defined before it is used. Every claim is labelled as a definition, a fact with a proof, a proposal, or an open question.

Two earlier artifacts exist. The extraction games in `games.tex` are one instance of what is defined here, and Section 12 says where they sit. The survey's frame of four coordinates, access point, criterion, budget and baseline, is recovered in Section 8, so nothing already written is lost, and one coordinate, the budget, is replaced by something more specific.

2 The problem the framework solves

A security definition for a learned model has to say what the adversary may do. For black-box access this is manageable: the adversary sends inputs and receives outputs, and one counts the queries. For white-box access it is not. The adversary holds the weights, and what it can learn from them depends on how training shaped them. A definition that covers that case seems to require a theory of the training map, the function from a corpus to a model. The map itself is public code that anyone can run. What no theory describes is what the map produces, and whether anything it produces can be obtained more cheaply than by running it.

Cryptography faced the same shape of problem with hash functions. No theory describes what SHA-256 does to its input, so the random oracle model replaces the hash function with an oracle that every party queries and no party can simulate, and proves theorems relative to that oracle. The generic group model does the same for group structure, and the Random Language Model of Brzuska and Couteau does it for language membership [9]. In each case the function is public, the hardness lies in reaching a *specified* output without paying the search or the walk, and the cost of an attack is counted in oracle queries.

The framework below makes that move for learning. The one object we cannot describe is the training map, so that is the one object the oracle idealizes. Inference is a computable function, analysis of weights is ordinary computation, and data is supplied by the game. The move has a precedent inside machine learning. Villa et al. formalize training as an abstract oracle inside a security game and prove one theorem there, on the tension between a classifier's usefulness and membership privacy [48]. The metering of that oracle, its role as the ambient model every game lives in, and the reading of its calls as the unit every party is priced in, are this document's additions, not theirs.

3 Notation

$\mathcal{X}, \mathcal{Y}$	input space and output space of a model
D	a corpus, a finite multiset of labelled examples from $\mathcal{X} \times \mathcal{Y}$
n	the security parameter, which scales the corpus, the model and the budgets
R	a recipe, Definition 1
w_t	the state of training after step t, a vector of weights
g_t	the gradient, or other per-step quantity, computed at step t
τ	a training trace, Definition 2
L	the learning oracle, Definitions 3 and 7
V	a view, Definition 5
$\mathcal{C}, \mathcal{A}$	the challenger and the adversary
steps	the within-recipe shorthand for metered cost, Definition 4

A *party* is any algorithm in a game. The challenger runs the game. The adversary tries to

win it. Other named parties, an auditor or a second adversary, appear where a game needs them. Everything is probabilistic polynomial time in n unless a game says otherwise.

4 Recipes and traces

The framework needs a name for a training procedure that is fixed and public but whose behaviour on a given corpus we do not claim to understand.

Definition 1 (Recipe)**.** A recipe R is a public description of a training procedure: an architecture, a loss, an optimizer, a batch rule, an initialization rule, a step count T, and any noise the procedure adds to its own computation. A recipe fixes a *step function* S that carries the current state w_{t-1} and a batch B_t to a per-step quantity g_t and a new state w_t.

A recipe is an object a paper would write in its experimental section. Stochastic gradient descent on a named architecture with a named learning rate is a recipe. Differentially private SGD, which clips and adds Gaussian noise to each gradient, is a different recipe, and the noise is inside it.

Definition 2 (Trace)**.** The trace of running recipe R on corpus D with coins r is the sequence

$$\tau = \big(w_0;\ (B_1, g_1, w_1);\ (B_2, g_2, w_2);\ \ldots;\ (B_T, g_T, w_T)\big),$$

where w_0 is the initial state drawn by the initialization rule, B_t the batch drawn by the batch rule, and $(g_t, w_t) = S(w_{t-1}, B_t)$. The final model is w_T. The *length* of the trace is T.

The trace is everything that happened during training, in order. A paper that releases a model releases w_T. A paper on gradient leakage studies some of the g_t. A proof-of-learning scheme asks a prover to exhibit a trace. These are all views of one object, which is the reason the trace and not the model is the oracle's output.

5 The learning oracle, passive form

Definition 3 (Passive learning oracle)**.** A game names a finite set of public recipes. The passive learning oracle L_{R} for a named recipe R is an oracle that, on input a corpus D, draws fresh coins r, runs R on D with r, and returns the trace τ. Two calls on the same corpus return independent traces. Parties call the oracle on the named recipes and do not author recipes of their own.

Definition 4 (Metered cost)**.** The cost of a call is the computational work of executing the recipe on the submitted corpus, in a fixed machine model, together with the size of that corpus. Within one recipe, whose step function is fixed, the shorthand for that work is the step count T, and this document prices calls in steps whenever the recipe is clear from context. A party's oracle cost is the total over its calls. A party also has its own running time, which is ordinary computation and is accounted separately.

Two restrictions inside these definitions carry weight, and each blocks a concrete attack on the accounting itself. Recipes are named by the game rather than submitted by parties, because an adversary permitted to author recipes defeats the unit twice over. A wrapping recipe whose single step runs all T steps of another recipe would collapse any step count to one, which is why cost is defined as the work of the step function and steps are only a shorthand. A laundering recipe whose initialization rule is a constant w^* would emit any precomputed weights as a legitimate trace at no metered cost, which is why the initialization rules of the named recipes are part of the game's

declaration and a game whose theorems depend on the meter names recipes whose initialization carries no party-chosen information.

What the oracle is for must be stated as plainly as what it is not for. The random oracle hides a random function, and that hidden randomness is what in-model lower bounds are made of. The learning oracle hides nothing. The recipe is public, its noise is declared, and a party with unbounded ordinary computation can compute anything computable from what its views grant it, trained models included. The oracle therefore manufactures no hardness, and no theorem in this document pretends otherwise. Every lower bound in this framework comes from what the views withhold, through the query-bounded machinery that already exists, statistical queries over the data, evaluation queries on the model, gradient queries on the loss. The oracle's role is what remains: it makes those query models views of one object, the trace, so that attacks and defences stated in different corners of the literature meet in one accounting, and it prices the honest party and the adversary in one unit so that ratio statements can be written at all.

Two claims about concrete training, call them *no skipping* and *no aiming*, are on this reading not facts of the model. They are the *transfer hypotheses*: the claims about concrete training that make the metered cost track real cost when a theorem leaves the model. No skipping asserts that for the recipe class in question, the content of a T-step trace is not obtainable below the work of the walk. Sequential functions are the fine-grained home of that shape of assumption [29], with one boundary, they price depth where the meter here is total work, and proof-of-learning is the literature that needs exactly this hypothesis and states it as an unproved requirement [33]. No aiming asserts that, for the class in question and the views a deployment actually exposes, there is no inverse map from desired model properties to weights cheaper than search by learning. Its scope is set by the views: evaluation access already suffices to reconstruct some declared classes, ReLU networks among them [16], so the hypothesis for such classes concerns what lies below the corresponding query counts, not the impossibility of reconstruction. Both are recipe-relative, both are falsifiable, and Section 11 collects the current evidence for and against each. A theorem proved in the model applies to a deployed system exactly to the extent the hypotheses hold for its recipe, which is the random oracle methodology's own structure, and the known failure mode of that methodology bounds what transfer may promise. Canetti, Goldreich and Halevi construct schemes secure in the Random Oracle Model with, in their words, "no implementation" that is secure [13]. Their counterexamples are contrived by design, and whether a learning analogue exists is open, so the application of an in-model theorem to a deployed system is an argument, not a corollary.

Given a model w_T from a view, evaluating it, differentiating it with respect to inputs or weights, and any other analysis of it are ordinary computation, charged to the party's running time and not to the meter. This is what makes white-box access expressible: the party holding weights may run singular value decomposition on a layer or probe features freely, and the floors that still bind it are those of the views it received, not of a prohibition the model cannot enforce.

6 Views

A game rarely hands a whole trace to a party. It hands over what the scenario says the party sees, and that is a function of the trace.

Definition 5 (View)**.** A view V is either a publicly known function of a trace, possibly randomized and possibly lossy, or a publicly specified stateful query interface to the trace or to the corpus distribution, with a stated query budget. A game assigns each party a view. When a trace τ is produced, the party receives $\mathsf{V}(\tau)$, or query access through the interface, and nothing else of τ.

The black-box wrapper of Example 1 and the statistical interface of Proposition 2 are views of the interface kind.

Views are game parameters, not oracle features. The oracle always produces the full trace. This is deliberate: if the oracle returned noisy gradients in one game and summary statistics in another it would be a different oracle per scenario, and there would be no single object to prove things about.

Example 1 (Views in use)**.**

- **Final model.** $\mathsf{V}(\tau) = w_T$. The released-weights setting.
- **Black-box access.** $\mathsf{V}(\tau)$ is a stateful wrapper that answers evaluation queries on w_T and counts them. The party never holds w_T. This is the API setting, and the count is the query budget.
- **Gradients without initialization.** $\mathsf{V}(\tau) = (g_1, \ldots, g_T)$. The party sees every gradient and not w_0.
- **Noisy channel.** $\mathsf{V}(\tau) = (g_t + \xi_t)_{t \le T}$ for independent noise ξ_t drawn by the view.
- **Sampled steps.** $\mathsf{V}(\tau) = (g_t)_{t \in I}$ for a random subset I of steps drawn by the view.
- **Summaries.** $\mathsf{V}(\tau)$ is the per-step mean and variance of the gradients.
- **Nothing.** $\mathsf{V}(\tau) = \perp$. The party learns that a trace was produced.

Fact 1 (Gradients plus initialization determine the model)**.** *Call a recipe* gradient-factoring *if its step function updates state through the per-step quantity alone, $w_t = U(w_{t-1}, g_t)$ for a public U, with any optimizer state carried inside w. Stochastic gradient descent, with or without momentum, and adaptive methods in their standard forms are gradient-factoring. For such a recipe, w_T is a function of w_0 and $(g_t)_{t \le T}$, so a party whose view contains the initial state and every per-step quantity can reconstruct the final model by its own ordinary computation, and a scenario in which a party "sees the gradients but not the model" is only meaningful when its view withholds something.*

Proof. $w_t = U(w_{t-1}, g_t)$ for each t, and the composition from w_0 is determined. The reconstruction applies the public U to quantities the view granted, which is ordinary computation and charges no oracle cost. □

Fact 1 is why the view is a named object. It forces a scenario to say what is withheld rather than to say informally that gradients were observed.

Recipe noise against view noise. Differentially private SGD adds noise inside the recipe. The learner trains on noisy gradients and the final model reflects it. A noisy channel adds noise inside a view. The learner trains on clean gradients and only the observer's copy is noisy. These are different objects. A theorem about one says nothing about the other, and a scenario must say which it means.

7 The learning oracle, interactive form

The passive oracle runs a whole training in one call with every part of the procedure honest. Federated learning, split learning and any setting where a party takes part in training while it runs are not of that shape. The party sends something into training at each round and receives something back, and it may deviate from what the recipe expects of it. The interactive oracle exposes the recipe's step as the unit of interaction.

Definition 6 (Ports)**.** A recipe may name *ports*. A port is a position in the step function where a quantity enters from outside or leaves to outside. Each port has an honest rule, the computation the recipe performs there when no external party is attached, and a *validity predicate*, the set of values the recipe accepts at that port.

Example 2 (Ports in two settings)**.** In federated averaging, each client is a port. Its outgoing quantity is the current global model, its incoming quantity is an update, its honest rule is local training on the client's data, and its validity predicate is whatever the server checks, for instance a norm bound. In split learning, the cut between the client half and the server half is a port in each direction. The client sends activations at the cut and receives gradients at the cut. The honest rule on each side is the forward or backward pass of that side.

Definition 7 (Interactive learning oracle)**.** The interactive learning oracle $\mathsf{L}^{\circ}_{\mathsf{R}}$ for a named recipe R with ports holds a training state. The game initializes it with a corpus for the recipe's data input and, where the recipe's honest port rules consume data, a corpus per port, all named by the game. A game *attaches* parties to ports before training starts. The oracle then runs one step per call. In each step it sends to every attached party the outgoing quantities of that party's ports, waits for the incoming quantities, applies the validity predicates, substitutes the honest rule at any port that is unattached or whose incoming value is invalid, and advances the state. The trace records the honest and the received values at every port. Each step's metered cost, Definition 4, is charged to the party the game names for it, by default the challenger.

Attaching a party to a port gives it two things at once. It *reads* the port's outgoing traffic, which is a view of the trace restricted to that port. And it *writes* the port's incoming traffic, which is an intervention in the trace. A party that should only observe is attached to a port with an honest rule it cannot override, which is a read without a write, and is the same as giving it the corresponding view of a passive trace.

Fact 2 (The passive oracle is a special case)**.** *Running* $\mathsf{L}^{\circ}_{\mathsf{R}}$ *for* T *steps with no party attached produces, after projecting the trace onto the honest values, the same distribution over traces as one call to* L_{R}*, and costs the same* T *steps.*

Proof. With no party attached the oracle substitutes the honest rule at every port at every step, which is exactly one run of the recipe, and the metered work is the same T steps of the same step function. The projection removes the per-port received-value fields, which the interactive trace carries and the passive trace does not. □

Definition 8 (Write budget)**.** A party attached to a port has a write budget: the number of steps at which it may send a value that differs from the honest rule, and, when the port's validity predicate carries a metric, a bound in that metric on how far the value may differ. A write budget of zero is read-only attachment.

Write budgets are the second kind of budget in the framework. The first kind, from Definition 4, is what a party spends on the oracle, a read budget. The second is what a party is permitted to change, a write budget. A poisoning fraction is a write budget at the corpus. A malicious federated client's permitted deviation is a write budget at its port. The prior systematization of this area separates adversaries by whether they can read or also write [40]. Here that separation is two numbers.

8 Parties and the two kinds of game

With the oracle in place, a game is a protocol among parties that all share the same oracle. The shape that recurs across the corpus is the following.

Definition 9 (Game skeleton)**.** A game names the following.

1. A recipe R and a distribution over corpora, from which the challenger draws.
2. For each party, its attachment to ports and its write budget (Definitions 6 and 8), and its view of any trace the challenger produces (Definition 5).
3. For each party, a read budget, in metered oracle cost and in submitted examples (Definition 4), and a bound on its running time.
4. Its declaration, the three-part object of Section 9.
5. An optional *secret*, a value drawn from a stated distribution with min-entropy at least h, given to some parties and withheld from others. When a party chooses the secret, the game states the distribution it must be drawn from, and the min-entropy requirement binds that distribution.
6. A *criterion*, of one of the two kinds below.
7. A *baseline*, the value of the criterion achieved without access, or without the secret, or by the best strategy that ignores the trace entirely.

The survey's frame had four coordinates: access point, criterion, budget, and baseline. Items 1 and 2 are the access point, now as attachments and views. Item 5 is the criterion. Items 3 and 4 split the old budget into a read budget and a write budget, and add the secret, which the old frame lacked and which Section 13 needs. Item 6 is the baseline.

Definition 10 (Distinguish criterion)**.** The challenger prepares two *worlds*, two settings of the corpus or of the ports that differ in a stated way. Either a party submits the two, or the challenger draws both. In both cases the challenger flips a coin b, runs the oracle in world b, hands each party its view, and a designated party outputs a guess b'. Its advantage is $|\Pr[b' = b] - \frac{1}{2}|$.

Definition 11 (Produce criterion)**.** The challenger runs the oracle once, hands each party its view, and a designated party outputs a value. The party wins if the value satisfies a stated predicate within a stated tolerance. Its advantage is its winning probability minus the baseline.

The two criteria are the search and decision forms that cryptography has always had. Membership inference, differential privacy and undetectability are distinguish games. Reconstruction, extraction and forgery are produce games. A backdoor game is both at once, and Section 13 composes them.

Fact 3 (Worlds must differ)**.** *In a distinguish game, if the two worlds induce the same distribution on the designated party's view, its advantage is zero for every strategy, and indistinguishability holds for every bound on every party. A distinguish game is therefore only informative when the game states how the worlds differ and the statement is checkable.*

Proof. The party's output is a function of its view and its coins, and both have the same distribution in either world. □

Fact 3 is the analogue of requiring the two challenge messages in an indistinguishability game to be distinct. It matters for the backdoor game, and Section 13 returns to it.

9 The adversary class

Definition 12 (View-generic adversary)**.** An adversary in a game is *view-generic*, below simply generic, if it interacts with the corpus, the training and the model only through the views and interfaces the game grants it, within their query budgets, obtains any further trace through oracle calls within its metered budget, and otherwise computes freely within its running time. Genericity is relative to the granted views, and a statement about a generic adversary names them.

This replaces an earlier dichotomy of generic against structural attacks that does not survive inspection, and the two examples that broke it now sit where they belong. Carlini, Jagielski and Mironov extract the parameters of a ReLU network using nothing but real-valued evaluation queries, reading piecewise linearity off second finite differences [16], and the extraction is polynomial in depth [12]. That attack is generic for any game whose granted view is an evaluation interface, and any lower bound for extraction under that view must sit at or below its query count. Gaussian elimination learns a noiseless parity from $O(d)$ raw samples. That attack is generic for a game that grants the raw corpus, and it is unavailable to an adversary whose granted view of the data is statistical queries, which is the view the parity floor is proven against. Neither attack bypasses anything. Each requires a view, and the informative classification of an attack is the pair of the weakest view it needs and the queries it spends there. What the earlier vocabulary called a structural attack decomposes into two honest cases: an attack that needs a richer view than a deployment grants, which is not an attack on that deployment, and a real-world procedure that violates a transfer hypothesis, which Proposition 1 addresses.

The second coordinate of the classification is the game's *declaration*, a named object with three parts: the class the recipe's outputs lie in, the target family with the unit that prices it, and the noise the data carries. Noise is a member and not a detail, because noiseless problems admit algebraic escapes, parity by Gaussian elimination among them, and the surviving positive low-degree implications hold only for noisy problems [32]. What a generic adversary may exploit through its views is fixed by the first part. A game that declares the recipe outputs ReLU networks of depth h admits the extraction attacks above at their query counts. A game that declares nothing beyond "a function" admits less and proves more, for fewer real systems. The declaration is the one place where knowledge of learning enters the framework, and it is part of what a game names, Definition 9.

Proposition 1 (Locating a real attack)**.** *Fix a game with a proven floor: every generic adversary, with granted views V and budgets B, has advantage at most ε. Suppose a deployed system implements the game so that the access it exposes corresponds to V and the resources the real attacker spends correspond to B, and suppose a real attack attains advantage above ε. Then at least one of three statements holds. The attacker's access exceeded V. The cost correspondence understated the attacker's spend against B. Or the granted views were simulated from below in reality, that is, some real procedure supplied the information content of a view or of an oracle call at less than its modeled price, which for the views built from training is a violation of the no-skipping or the no-aiming transfer hypothesis for the deployed recipe.*

Proof. The floor quantifies over every algorithm interacting through V within B, with no further restriction, so an attack whose access, spend and information sources all match the model is such an algorithm and is bound by ε. An attack exceeding ε therefore fails the correspondence in at least one of its three clauses, access, cost, or the modeled price of what the views delivered, and the three listed statements are those clauses. □

The proposition is an audit, and that is its value. It does not say a deployed break is impossible, it says where the break must live, and each location is informative. The first is a threat-model error. The second is an accounting error. The third, when the underpriced object is a trained model or a trace, is a procedure that trains or aims cheaper than the recipe, which is a learning-theoretic discovery in the sense the win-win argument of Degwekar, Nakkiran and Vaikuntanathan gives the phrase [22], an argument this framework shares in shape and does not import as a theorem.

10 Cost, and the ratio between parties

Each party's cost has three parts: steps caused, examples submitted, and running time. A security statement bounds the adversary's advantage for every adversary whose cost lies under a stated threshold. The statements the survey cares about most set the threshold relative to an honest party's cost. Fine-grained cryptography writes that relation as an exponent [9, 37]: the adversary is permitted up to the honest cost raised to $1+\delta$, and the question is the largest δ at which security holds. In this framework the honest cost is a step count, which is dimensionless, so raising it to a power is well defined. The three cautions in `games.tex`, Section 6, on the three incompatible conventions for δ and on the ceiling of Barak and Mahmoody being a query bound [6], apply unchanged.

One consequence is specific to the oracle. Every bound proved here is a bound on oracle steps, not on time. Barak and Mahmoody's adversary against Merkle puzzles makes few oracle queries and runs efficiently only in a relativized world where P = NP [6]. The same will hold of any adversary exhibited by a lower-bound argument in this model. A theorem here should say "steps" where it means steps.

11 The price of the search

Section 5 names the two transfer hypotheses and states that every floor in this framework comes from what the views withhold. This section grounds both hypotheses in the sharpest current results and reports one experiment. Everything here carries its epistemic status inline, because the ground shifts: two of the units this section could have been built on two years ago are now known to be algorithm-relative artifacts.

11.1 The price of aiming

The no-aiming hypothesis says that, below the known query counts, learning is the only search over model space. Its in-model counterpart is a floor against a view-generic adversary that must produce a model with a target property, and the floor depends on the view of the data the game grants.

For recipes whose steps are statistical queries, the floor is unconditional. A hidden k-sparse parity requires $d^{\Omega(k)}$ statistical queries at any useful tolerance, and this is the number Barak et al. describe SGD as approaching [5]. One hypothesis connects that floor to gradient recipes, and its status must be stated. A fresh-batch gradient step of batch size B evaluates a correlational statistical query to tolerance on the order of $B^{-1/2}$, up to concentration, so a T-step fresh-batch recipe reads as a correlational-query algorithm with T queries at that tolerance. The reading is standard and is how Barak et al. compare SGD to the SQ limit [5], and it is not a theorem. Dandi et al. state the inheritance for reused batches as an expectation explicitly [20]. Every use of a query floor for a gradient recipe in this framework routes through this simulation and carries its status.

For Gaussian single-index models the current sharp single number is the generative exponent k^*. Any statistical-query algorithm using polynomially many queries, under the coefficient condition its theorem states, needs a sample-size parameter $n \gtrsim d^{k^*/2}$ to output a direction with nontrivial overlap, unconditionally within the class [19]. Throughout this section n is the sample count, as in the sources, and the ambient security parameter does not appear. One caveat travels with that citation and it runs in the safe direction: the authors state that their theorem primarily measures the detection threshold, and the statistical-query estimation threshold is open, so the true price of recovery can only be higher than the floor quoted here. For multi-index models the same quantity governs subspace recovery, with an unconditional estimation upper bound at the matching rate and a low-degree lower bound [18]. Any reading of the low-degree half as polynomial-time hardness must carry the refutation of the quasi-polynomial low-degree conjecture and its surviving positive counterpart [10, 32].

Two units this section does not use, and why. The information exponent prices a single algorithm, online SGD from a random start [2], and the leap's lower bound holds for correlational statistical queries only, with its SGD side an open conjecture [1]. Both thresholds collapse under batch reuse [20], and the collapse is not an anomaly: reuse reaches the information-theoretic rate for polynomial links [38], a learning-rate schedule induces the same transition without carrying any sample across iterations, though each sample is queried twice within its own iteration [47], and full-batch descent separates from one-pass descent by a logarithmic factor within the analyzed dynamics, with the separation proved for a truncated variant of the target and provably absent for the plain one [36]. What survives this wave is the generative exponent, whose defining invariance is exactly closure under the transformations reuse exploits [19]. Proven, all of it, in the classes named.

The algebraic escape calibrates the floor rather than refuting it. Noiseless parity is solvable by Gaussian elimination from $O(d)$ raw samples. That algorithm is generic for a game that grants the raw corpus and unavailable to one whose view of the data is statistical queries, which is the view the floor is proven against. Parity is therefore the case where the view classification of Section 9 is not hypothetical. The same task is easy under one view and $d^{\Omega(k)}$-hard under another, and the gap between the two views is a named theorem about what learning-shaped access cannot do.

11.2 The price of skipping is recipe-relative

The no-skipping hypothesis as stated in Section 5 prices the walk of one recipe. The literature now forces exactly that scoping, and the force is worth exhibiting, because a stronger reading of the hypothesis is false.

The walk's length is a property of the pair recipe and query type, not of the task. The batch-reuse and learning-rate results above shorten the same walk to the same target by changing the recipe slightly [20, 38, 47], and empirically the compute needed to reach a fixed language-model loss has halved roughly every eight months, with a wide stated uncertainty of five to fourteen months, across a decade of recipe drift [30]. So a task-level no-skipping claim is refuted, and a recipe-level one is the strongest defensible form. Within a fixed recipe, the compute-to-loss frontier is a theorem in solvable models: for power-law random features under one-pass stochastic gradient descent the loss curve is exact and the compute-optimal frontier is a derived object [41], which is the theorem-shaped form of the empirical scaling laws [35, 31], whose fitted constants are themselves contested [7]. The mechanism literature makes the exponent explicitly algorithm-dependent: allowing feature learning nearly doubles the compute exponent on hard tasks [8], and a good data-pruning rule changes the law's form outright [43]. None of these results claims a lower bound over all algorithms, and no such bound exists. Open, and it is the open problem under every compute-pricing claim in this

survey.

Two consequences are definitional in the oracle model and consistent with the empirical record. First, amortization is not skipping. Distillation reaches a target cheaper only below a student-dependent compute threshold, and only when a stronger teacher already exists or its training cost is amortized across other uses, so the teacher's training was an oracle call somebody paid for [11]. In the model, a party that builds a derivative model builds it from views of prior traces, so the cost sits upstream in the ledger. There is no free trace, only amortized traces. Second, the price of a view is not the price of the trace. Proof-of-learning verification accepts a lossy view of the trace, and the published forgeries attack that view, succeeding without paying the walk [25], while exact reproduction of a trace remains empirically out of reach [45]. A sound proof-of-learning is precisely a view whose price equals the trace's, and whether one exists is the open problem that literature keeps encountering from inside.

11.3 An experiment: the measured exponent of the search

The transfer hypotheses assert that no cheaper route than the recipe exists, and they predict nothing about how the recipe's own cost grows. The experiment below therefore tests alignment, not the hypotheses. It measures how the walk lengthens for one recipe on the problem whose query floor is unconditional, and asks whether anything in the measurement contradicts that floor. Sparse parity is the cleanest instance, because the statistical-query floor is unconditional and the statistical baseline of $\Theta(k \log d)$ samples makes the gap between information and computation visible. Barak et al. report the phase transition and its scaling [5]. The run below matches their Figure-2 width, batch and learning rate, with one stated difference, momentum 0.9 where they run SGD with no auxiliary state, over a dimension range extending beyond their fitted grid of $n \leq 30$, into the range where their own plots show the exponent drifting, so that the survey's numbers are its own.

A two-layer ReLU network of width 100 is trained by minibatch SGD with momentum 0.9, batch 128, learning rate 0.01, on fresh samples per step, to learn a hidden 3-sparse parity of d bits, for d from 20 to 60, 8 seeds per dimension. Test accuracy is checked every 25 steps, so every recorded cost is a multiple of 25. The cost recorded is oracle steps to reach 90% test accuracy. All 40 of 40 runs reached it. The median cost rises from 1025 steps at $d = 20$ to 9337.5 at $d = 60$, and the log-log fit across dimensions gives

$$t_c \;\propto\; d^{2.02},$$

Figure 1. The fit spans five dimensions covering a factor of three, the consecutive-point slopes of the medians range from 0.8 to 3.0, and no goodness-of-fit statistic is claimed, so the fitted exponent summarizes a short power-law stretch and is not a three-digit measurement. Three further sentences fix what it is and is not. It sits in the power-law family $t_c \propto d^{\alpha k}$ that Barak et al. document, whose exponent depends on the architecture, the batch and the regime, and whose own scaling plots show it worsening at larger d [5]. It is consistent with the query floor through the simulation above, since the floor constrains the product of queries and squared tolerance and fixes no step exponent at a given batch. It does not measure the floor, and no experiment can, since a floor quantifies over algorithms. The convergence times are heavy-tailed across seeds at fixed dimension, which matches the initialization sensitivity Barak et al. document. A second regime changes three hyperparameters at once, batch 8, width 512 and learning rate 0.1, so it isolates no single cause, and its wall is sharper than any fit. The median over three seeds at $d = 30$ is 6940 steps, and none of nine runs at $d \in \{60, 90, 120\}$ reached threshold within 300000 steps, a greater than forty-fold increase against a doubling of dimension. Empirical, all of it, in the configurations stated.

What the experiment does and does not show. It shows one recipe's cost growing as a power of the dimension, in the regime where the recipe's steps read as noisy correlational queries. It does

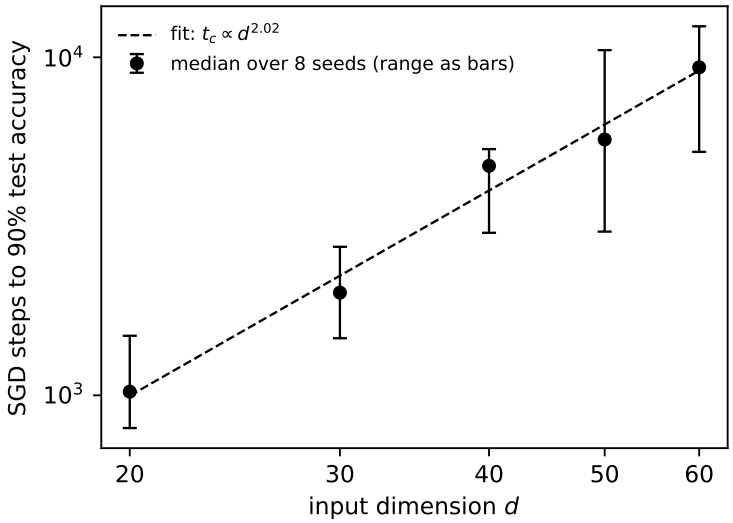


Figure 1: Median steps of minibatch SGD with momentum to 90% test accuracy against input dimension for a hidden 3-sparse parity, log-log, 8 seeds per point, bars spanning the seed range. The fitted exponent 2.02 summarizes this recipe's cost growth over the plotted range.

not measure the floor itself, since no experiment can quantify over algorithms, and it says nothing about adversaries granted richer views, for which noiseless parity is easy. It is one recipe's cost curve, consistent with the no-aiming hypothesis for the class that trains, and nothing more.

12 The corpus, mapped onto the framework

This section places the families the survey covers into Definition 9, to show that nothing in the framework is particular to one of them.

Membership inference. Distinguish. The worlds differ by one record in the corpus. The adversary's view is the final model, or black-box access. The baseline is the blind attack, which Das et al. show beats published attacks on eight benchmarks [21]. A shadow-model attack [42, 15] is generic and costs $N \cdot T$ steps for N shadow models. The verifier of Zhang et al., who cannot sample the null hypothesis because doing so "would require training new large models" [49], is an auditor with a read budget of zero steps.

Membership inference from gradients. Distinguish, with the adversary's view set to gradients, Example 1. Fact 1 says which lossy view the scenario must name.

Gradient inversion. Produce. The view is gradients or activations at a port, and the criterion is reconstruction within a tolerance. The reconstruction-robustness definition of Balle, Cherubin and Hayes already supplies the criterion and the oblivious baseline [3], and the gradient-inversion literature, for instance [4], supplies attacks with no stated criterion. Passive observation of a federated round is a read-only attachment. A malicious server is a write.

Computational differential privacy. Distinguish, with the worlds drawn by the challenger as

neighbouring corpora and the distinguisher bounded [39]. Recipe noise, Section 6, is where the mechanism lives.

Evasion. Produce. The criterion is a label change inside a ball around a given input. The baseline is random perturbation at the same radius, and the geometric results bound how near the nearest error must lie for a model with nontrivial error rate [27]. Access is a view: black-box is the metered wrapper, white-box is the final model, and transfer is a shadow model plus the adversary's own gradient computation. The number of gradient steps an attack is allowed, which the evaluation guidance of Carlini et al. declines to bound [14] and which Suya et al. use to rank attacks [46], is the adversary's running time in Definition 12.

Extraction. Produce. The games of `games.tex` are this case with the view set to black-box access, and their two-party pricing is Section 10.

Poisoning. A write budget at the corpus. Availability poisoning is produce, with the criterion an accuracy drop. Certified defences bound the drop for every write within the budget [44]. Robust estimation in high dimensions is the case where the defender is polynomial time and the writer is unbounded except in the fraction it may change [23].

Proof-of-learning. Produce. The criterion is a trace that a verifier accepts. The honest prover's trace comes from T oracle steps, and forging is producing an accepted trace in fewer. Jia et al. require that forging cost at least as much as honest training and conjecture rather than prove it [33]. In this framework the requirement is a bound on the forger's steps relative to T, and the attacks that break their scheme succeed against the verifier's lossy view of the trace [25], which is the view-against-trace gap Section 11.2 prices.

Watermarking. Distinguish, with a secret as the hinge: the detector holds the key and the distinguisher does not, as in Christ, Gunn and Zamir [17].

Filtering and auditing. Distinguish, where the designated party is an auditor whose read budget is stated relative to the model's own cost.

13 The backdoor game, as far as it is fixed

This is the one game that composes both criteria. It is stated here to the extent it has been agreed, with the open decisions marked.

Fixed. Three roles. A *planter* intervenes during training through a write budget at a port or at the corpus. An *auditor* observes through a view and a read budget and outputs a world guess, the distinguish criterion. An *activator* holds a secret drawn with min-entropy h, chosen by the planter, and tries to elicit a stated behaviour from the final model, the produce criterion. The activator's baseline is the success of an activator without the secret, which is the non-replicability property of Goldwasser, Kim, Vaikuntanathan and Zamir [28]. The auditor never holds the secret, because an auditor given the secret detects by running the activator, so undetectability is meaningful only against auditors without it.

Fixed. The planter is generic, Definition 12. The theory constructions in the corpus all give the planter control of training itself and all place cryptographic structure inside the model. Goldwasser, Kim, Vaikuntanathan and Zamir achieve black-box undetectability from digital signatures and

white-box undetectability for two specific training paradigms from continuous LWE and sparse PCA [28]. Kalavasis et al. achieve white-box undetectability for pipelines that end in an obfuscation step [34]. Draguns et al. compile a hash-based digital locker into a transformer and achieve unelicitability, which they state plainly is not white-box undetectability [24]. A planter that only writes to the corpus and faces an honest recipe is the case those constructions do not cover.

Fixed. The auditor performs detection, not reconstruction. The question is whether some intervention occurred, and the auditor is not asked to say which.

Open 1 (Non-triviality). By Fact 3, a planter whose writes leave the distribution of the auditor's view unchanged is undetectable and has done nothing. The game needs a condition on world 1 that rules this out. The activator's success is one such condition. If the game is to be stated without requiring the backdoor to work at inference, another condition must replace it, for instance a minimum write budget together with a declared difference in the label distribution of the written examples. Which condition to use is not decided.

Open 2 (Where the planter writes). The planter may write at the corpus, at a training port, or by replacing the data distribution itself. These give different games, and the last puts no fraction in the game at all. Which of them the survey's backdoor section treats first is not decided.

The corpus-level half of the game's first theorem is a specialization of existing machinery, and it is worth stating because it prices a party this literature leaves unpriced.

Proposition 2 (Undetectability against a query-bounded corpus auditor)**.** *Fix the backdoor game in which world 0 draws the corpus from a distribution D_0 and world 1 draws it from a distribution the planter selects out of a family $\mathcal{S}$, and grant the auditor, as its view of the data, a* VSTAT(m) *interface to the corpus distribution with a query budget. This is the decision problem between D_0 and $\mathcal{S}$ in the statistical algorithms model, and the statistical-dimension lower bound of Feldman et al. prices it: an auditor with fewer queries than the statistical dimension at the matching discrimination parameter fails to decide the world at their stated success level [26]. In their planted bipartite k-clique instantiation, on graphs of size N with $k \le N^{1/2-\delta}$ for constant $\delta > 0$: deciding requires $N^{\Omega(\log r)}$ queries to* VSTAT($N^2/(rk^2)$) *for every $r > 0$, and in particular $N^{\Omega(\log N)}$ queries to* VSTAT($N^{2-\delta}/k^2$).

Proof. Specialization. The auditor's task, as fixed, is the model and the problem of Feldman et al., and the parameters are their Theorem 2.9 [26]. Nothing beyond the specialization is claimed. □

Four boundaries keep the proposition honest. The VSTAT(m) interface is an oracle to the population distribution whose answer tolerance models the averages of roughly m independent samples, which is how Feldman et al. motivate it, so the correspondence to an auditor holding a corpus of about m records is the model's own reading and not a theorem. It prices an auditor whose access to the data is statistical averages, and says nothing about an auditor who reads individual records, for whom the raw-sample view makes the problem different. It does not touch the auditor who views the trained model rather than the corpus, and that transfer is open. And it does not assert that a generic planter can meet a non-triviality condition with a clique-shaped poison, which is Open 1. The proposition instantiates the distribution-replacement variant of the planter, the third option of the open question on where the planter writes, and its floor comes entirely from the view restriction, which is the division of labour Section 5 states.

Open 3 (The model-level auditor). Whether Proposition 2 transfers to an auditor viewing the final model, rather than the corpus, is not known, and it is the half of the first theorem that would price backdoor detection as deployed defences practice it.

14 What is deferred, and what remains open about the oracle

Deferred 1 (The criterion for generation). The produce criterion assumes a predicate the challenger can evaluate. For a language model asked to be harmless, the predicate is not something we can write down. Villa et al. idealize that predicate as a ground truth and bound the gap to a computable judge [48]. This framework does not yet include that second idealization, and a game about generation needs it.

Deferred 2 (Classification). The tolerance in a produce criterion assumes a metric on outputs. For classifiers under the zero-one metric, the propositions in `games.tex` that rest on a Lipschitz constant do not apply, and the produce games need a separate treatment.

Open 4 (Declaration). Section 9 fixes that what the adversary may exploit is what the game declares about the recipe's output. The framework does not yet say what a declaration may contain. Too little, and every theorem is about random functions. Too much, and the declaration is the theory of learning the oracle was introduced to avoid. The declaration must also assert that the recipe has no known shortcut, because the transfer hypotheses are false where one exists: a convex recipe such as linear regression has a closed-form endpoint, so the content of its trace is obtainable with none of its steps. The idealization is only credible for recipe classes in the no-known-shortcut regime, as the random oracle model is only credible for hash functions with no known structure. Where that line sits is the main open question about the oracle itself.

Open 5 (Uninstantiability). Random-oracle proofs are known to have instantiations for which the proved theorem fails, by the construction of Canetti, Goldreich and Halevi [13]. Whether the analogous phenomenon arises when the learning oracle is replaced by a concrete recipe is not known. A learning analogue of their diagonalization, a game secure in this model that every concrete recipe implements insecurely, would put the same known bound on this methodology that theirs put on the random oracle.

References

[1] Emmanuel Abbe, Enric Boix-Adserà, and Theodor Misiakiewicz. Sgd learning on neural networks - leap complexity and saddle-to-saddle dynamics. arXiv:2302.11055, 2023.

[2] Gerard Ben Arous, Reza Gheissari, and Aukosh Jagannath. Online stochastic gradient descent on non-convex losses from high-dimensional inference. arXiv:2003.10409, 2021.

[3] Borja Balle, Giovanni Cherubin, and Jamie Hayes. Reconstructing training data with informed adversaries. In *IEEE S&P*, 2022.

[4] Mislav Balunović, Dimitar I. Dimitrov, Nikola Jovanović, and Martin Vechev. Lamp: Extracting text from gradients with language model priors. In *NeurIPS*, 2022.

[5] Boaz Barak, Benjamin L. Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Hidden progress in deep learning - sgd learns parities near the computational limit. arXiv:2207.08799, 2022.

[6] Boaz Barak and Mohammad Mahmoody-Ghidary. Merkle puzzles are optimal — an $o(n^2)$-query attack on any key exchange from a random oracle. In *CRYPTO*, 2009.

[7] Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You. Chinchilla scaling: A replication attempt. arXiv:2404.10102, 2024.

[8] Blake Bordelon, Alexander Atanasov, and Cengiz Pehlevan. How feature learning can improve neural scaling laws. In *ICLR*, 2025.

[9] Chris Brzuska and Geoffroy Couteau. On building fine-grained one-way functions from strong average-case hardness. In *EUROCRYPT*, 2022.

[10] Rares-Darius Buhai, Jun-Ting Hsieh, Aayush Jain, and Pravesh K. Kothari. The quasi-polynomial low-degree conjecture is false. In *FOCS*, 2025.

[11] Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, and Russ Webb. Distillation scaling laws. In *ICML*, 2025.

[12] Isaac A. Canales-Martínez, Jorge Chávez-Saab, Anna Hambitzer, Francisco Rodríguez-Henríquez, Nitin Satpute, and Adi Shamir. Polynomial time cryptanalytic extraction of neural network models. In *EUROCRYPT*, 2024.

[13] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited. In *STOC*, 2004.

[14] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Mądry, and Alexey Kurakin. On evaluating adversarial robustness. arXiv:1902.06705, 2019.

[15] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramèr. Membership inference attacks from first principles. In *IEEE S&P*, 2022.

[16] Nicholas Carlini, Matthew Jagielski, and Ilya Mironov. Cryptanalytic extraction of neural network models. In *CRYPTO*, 2020.

[17] Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. arXiv:2306.09194, 2023.

[18] Alex Damian, Jason D. Lee, and Joan Bruna. The generative leap - sharp sample complexity for efficiently learning gaussian multi-index models. arXiv:2506.05500, 2025.

[19] Alex Damian, Loucas Pillaud-Vivien, Jason D. Lee, and Joan Bruna. Computational-statistical gaps in gaussian single-index models. arXiv:2403.05529, 2024.

[20] Yatin Dandi, Emanuele Troiani, Luca Arnaboldi, Luca Pesce, Lenka Zdeborová, and Florent Krzakala. The benefits of reusing batches for gradient descent in two-layer networks - breaking the curse of information and leap exponents. arXiv:2402.03220, 2024.

[21] Debeshee Das, Jie Zhang, and Florian Tramèr. Blind baselines beat membership inference attacks for foundation models. In *DATA-FM workshop @ ICLR*, 2025.

[22] Akshay Degwekar, Preetum Nakkiran, and Vinod Vaikuntanathan. Computational limitations in robust classification and win-win results. In *COLT*, 2019.

[23] Ilias Diakonikolas, Gautam Kamath, Daniel M. Kane, Jerry Li, Ankur Moitra, and Alistair Stewart. Robust estimators in high dimensions without the computational intractability. arXiv:1604.06443, 2016.

[24] Andis Draguns, Andrew Gritsevskiy, Sumeet Ramesh Motwani, Charlie Rogers-Smith, Jeffrey Ladish, and Christian Schroeder de Witt. Unelicitable backdoors in language models via cryptographic transformer circuits. arXiv:2406.02619, 2024.

[25] Congyu Fang, Hengrui Jia, Anvith Thudi, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning is currently more broken than you think. arXiv:2208.03567, 2023.

[26] Vitaly Feldman, Elena Grigorescu, Lev Reyzin, Santosh S. Vempala, and Ying Xiao. Statistical algorithms and a lower bound for detecting planted cliques. In *JACM*, 2017.

[27] Justin Gilmer, Luke Metz, Fartash Faghri, Samuel S. Schoenholz, Maithra Raghu, Martin Wattenberg, and Ian Goodfellow. The relationship between high-dimensional geometry and adversarial examples (adversarial spheres). arXiv:1801.02774, 2018.

[28] Shafi Goldwasser, Michael P. Kim, Vinod Vaikuntanathan, and Or Zamir. Planting undetectable backdoors in machine learning models. arXiv:2204.06974, 2022.

[29] Jiaxin Guan and Hart Montgomery. On sequential functions and fine-grained cryptography. In *CRYPTO*, 2024.

[30] Anson Ho, Tamay Besiroglu, Ege Erdil, David Owen, Robi Rahman, Zifan Carl Guo, David Atkinson, Neil Thompson, and Jaime Sevilla. Algorithmic progress in language models. arXiv:2403.05812, 2024.

[31] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models (chinchilla). arXiv:2203.15556, 2022.

[32] Jun-Ting Hsieh, Daniel M. Kane, Pravesh K. Kothari, Jerry Li, Sidhanth Mohanty, and Stefan Tiegel. Rigorous implications of the low-degree heuristic. In *STOC*, 2026.

[33] Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning: Definitions and practice. In *IEEE S&P*, 2021.

[34] Alkis Kalavasis, Amin Karbasi, Argyris Oikonomou, Katerina Sotiraki, Grigoris Velegkas, and Manolis Zampetakis. Injecting undetectable backdoors in obfuscated neural networks and language models. arXiv:2406.05660, 2024.

[35] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. arXiv:2001.08361, 2020.

[36] Filip Kovačević, Hong Chang Ji, Denny Wu, Mahdi Soltanolkotabi, and Marco Mondelli. Full-batch gradient descent outperforms one-pass sgd - sample complexity separation in single-index learning. arXiv:2602.02431, 2026.

[37] Rio LaVigne, Andrea Lincoln, and Virginia Vassilevska Williams. Public-key cryptography in the fine-grained setting. In *CRYPTO*, 2019.

[38] Jason D. Lee, Kazusato Oko, Taiji Suzuki, and Denny Wu. Neural network learns low-dimensional polynomials with sgd near the information-theoretic limit. arXiv:2406.01581, 2024.

[39] Ilya Mironov, Omkant Pandey, Omer Reingold, and Salil Vadhan. Computational differential privacy. In *CRYPTO*, 2009.

[40] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael P. Wellman. Sok: Security and privacy in machine learning. In *IEEE EuroS&P*, 2018.

[41] Elliot Paquette, Courtney Paquette, Lechao Xiao, and Jeffrey Pennington. 4+3 phases of compute-optimal neural scaling laws. arXiv:2405.15074, 2024.

[42] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. arXiv:1610.05820, 2017.

[43] Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari S. Morcos. Beyond neural scaling laws, beating power law scaling via data pruning. In *NeurIPS*, 2022.

[44] Jacob Steinhardt, Pang Wei Koh, and Percy Liang. Certified defenses for data poisoning attacks. arXiv:1706.03691, 2017.

[45] Mohamed Suliman, Anisa Halimi, Swanand Ravindra Kadhe, Nathalie Baracaldo, and Douglas Leith. Towards a re-evaluation of data forging attacks in practice. arXiv:2411.05658, 2025.

[46] Fnu Suya, Anshuman Suri, Tingwei Zhang, Jingtao Hong, Yuan Tian, and David Evans. Sok: Pitfalls in evaluating black-box attacks. arXiv:2310.17534, 2024.

[47] Konstantinos Christopher Tsiolis, Alireza Mousavi-Hosseini, and Murat A. Erdogdu. From information to generative exponent, learning rate induces phase transitions in sgd. arXiv:2510.21020, 2025.

[48] Federico Villa, F. Betül Durak, Tadayoshi Kohno, Tapdig Maharramli, and Franziska Roesner. Extending the formalism and theoretical foundations of cryptography to ai. arXiv:2603.02590, 2026.

[49] Jie Zhang, Debeshee Das, Gautam Kamath, and Florian Tramèr. Position: Membership inference attacks cannot prove that a model was trained on your data. arXiv:2409.19798, 2025.